



杭州电子科技大学
HANGZHOU DIANZI UNIVERSITY

电子信息技术 虚拟仿真综合实践

实验二：基于FPGA-ROM的
LCD彩色图像显示控制器设计

2024

一、ROM简介

ROM是只读存储器（Read-Only Memory），是一种只能读出事先所存数据的固态半导体存储器。

ROM所存数据稳定，断电后所存数据也不会改变；其结构较简单，读出较方便，因而常用于存储各种固定程序和数据。

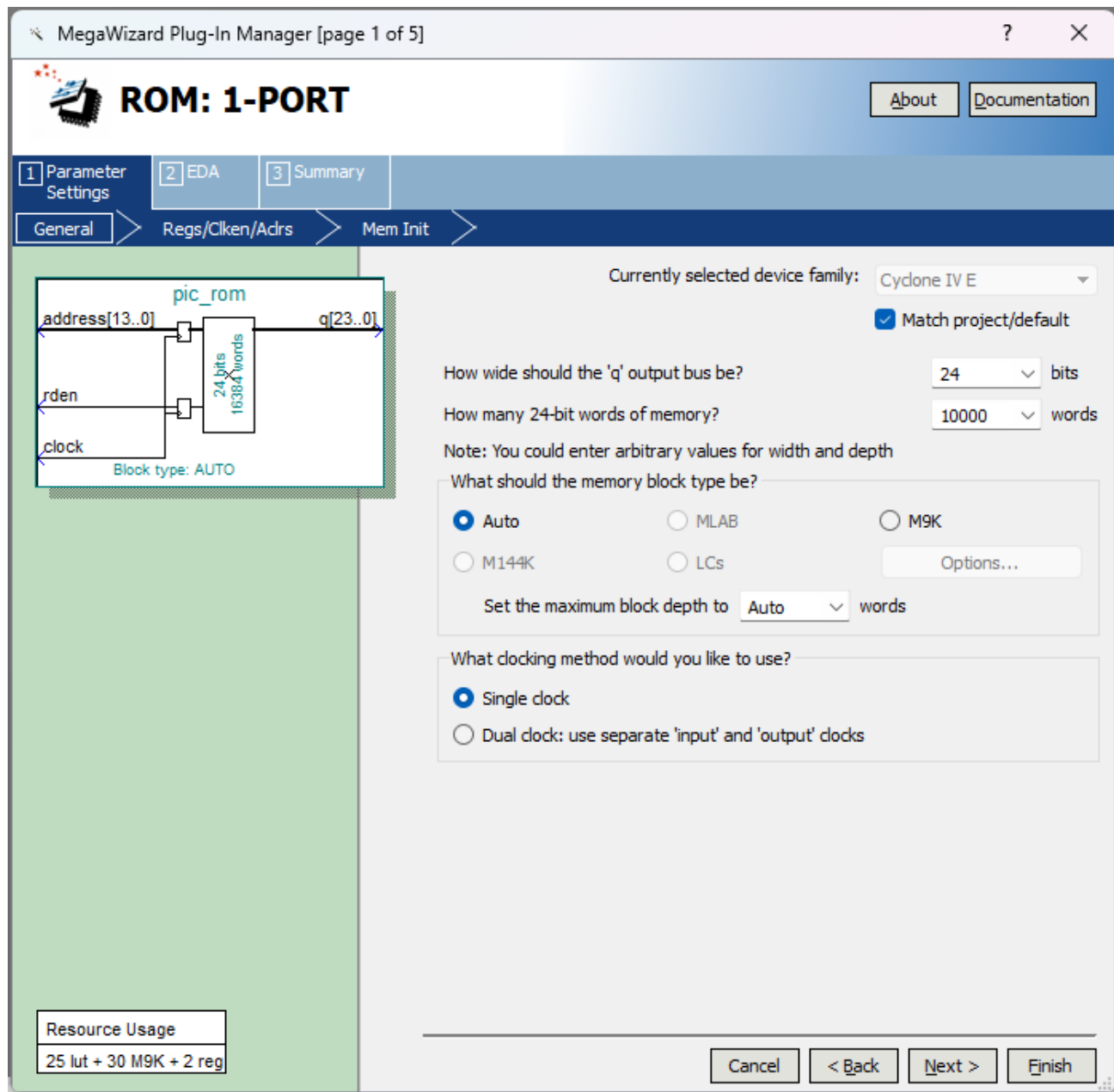
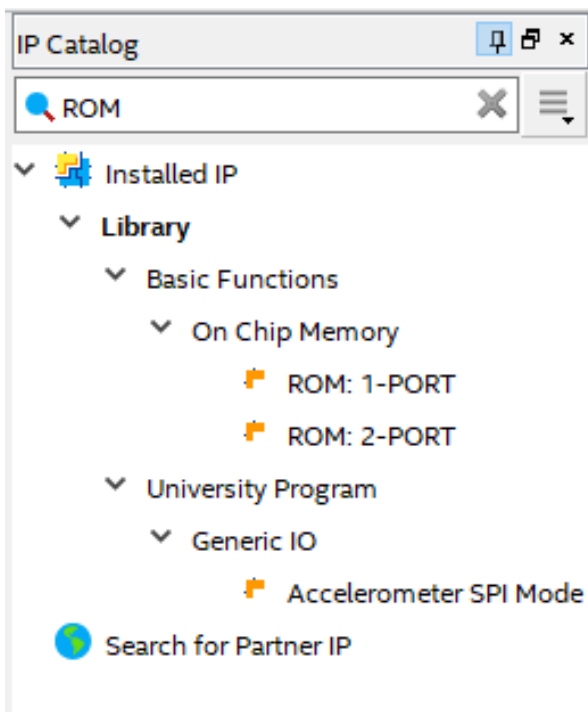
Resource		Device							
		10CL006	10CL010	10CL016	10CL025	10CL040	10CL055	10CL080	10CL120
Logic Elements (LE)		6,272	10,320	15,408	24,624	39,600	55,856	81,264	119,088
M9K Memory	Block	30	46	56	66	126	260	305	432
	Capacity (Kb)	270	414	504	594	1,134	2,340	2,745	3,888
18 × 18 Multiplier		15	23	56	66	126	156	244	288
PLL		2	2	4	4	4	4	4	4
Clock		20	20	20	20	20	20	20	20
Maximum I/O		176	176	340	150	325	321	423	525
Maximum LVDS		65	65	137	52	124	132	178	230

LCD显示屏： 800*480*16 (RGB565) /1024=6000Kbits

分辨率100*100, 156Kbits

一、ROM简介

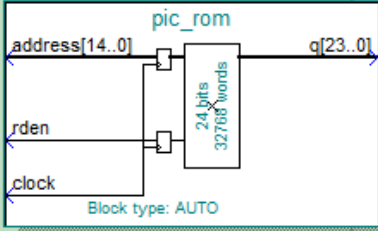
Quartus II软件中提供了ROM IP核，支持单端口ROM IP核和双端口ROM IP核。



一、ROM简介

MegaWizard Plug-In Manager [page 2 of 5]

? X



Block type: AUTO

ROM: 1-PORT

About Documentation

1 Parameter Settings 2 EDA 3 Summary

General > Regs/Clen/Adrs > Mem Init >

Which ports should be registered?

☐ 'data' input port

☒ 'address' input port

☐ 'q' output port

Create one clock enable signal for each clock signal.

☐ Note: All registered ports are controlled by the enable signal(s) More Options...

☐ Create byte enable for port A

What is the width of a byte for byte enables? bits

☐ Create an 'adr' asynchronous clear for the registered ports More Options...

☒ Create a 'rden' read enable signal

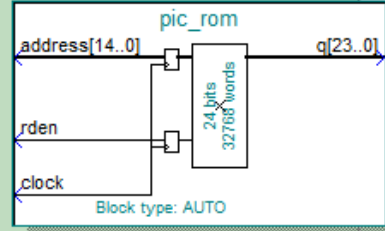
Resource Usage

52 lut + 2 M9K + 48 M9K + 3 reg

Cancel < Back Next > Finish

MegaWizard Plug-In Manager [page 3 of 5]

? X



Block type: AUTO

ROM: 1-PORT

About Documentation

1 Parameter Settings 2 EDA 3 Summary

General > Regs/Clen/Adrs > Mem Init >

Do you want to specify the initial content of the memory?

☐ No, leave it blank

☐ Initialize memory content data to XX..X on power-up in simulation

☒ Yes, use this file for the memory content data

(You can use a Hexadecimal (Intel-format) File [.hex] or a Memory Initialization File [.mif])

Browse...

File name:

The initial content file should conform to which port's dimensions?

☐ Allow In-System Memory Content Editor to capture and update content independently of the system clock

The 'Instance ID' of this ROM is:

Resource Usage

52 lut + 2 M9K + 48 M9K + 3 reg

Cancel < Back Next > Finish

一、ROM简介

MegaWizard Plug-In Manager [page 5 of 5]

ROM: 1-PORT [About](#) [Documentation](#)

1 Parameter Settings 2 EDA 3 Summary

pic_rom

address[14..0] q[23..0]

rden

clock

Block type: AUTO

24 bits
32768 words

Turn on the files you wish to generate. A gray checkmark indicates a file that is automatically generated, and a green checkmark indicates an optional file. Click Finish to generate the selected files. The state of each checkbox is maintained in subsequent MegaWizard Plug-In Manager sessions.

The MegaWizard Plug-In Manager creates the selected files in the following directory:
D:\FPGA\ROM_PIC\

File	Description
☒ pic_rom.v	Variation file
☐ pic_rom.inc	AHDL Include file
☐ pic_rom.cmp	VHDL component declaration file
☐ pic_rom.bsf	Quartus Prime symbol file
☒ pic_rom_inst.v	Instantiation template file
☒ pic_rom_bb.v	Verilog HDL black-box file

Resource Usage

52 lut + 2 M9K + 48 M9K + 3 reg

Cancel < Back Next > Finish

```
module pic_rom (  
    address,  
    clock,  
    rden,  
    q);  
  
    input    [14:0] address;  
    input    clock;  
    input    rden;  
    output   [23:0] q;  
  
    `ifndef ALTERA_RESERVED_QIS  
    // synopsys translate_off  
    `endif  
  
        tri1    clock;  
        tri1    rden;  
  
    `ifndef ALTERA_RESERVED_QIS  
    // synopsys translate_on  
    `endif
```

一、ROM简介

mif文件的创建主要方法：

- ◆ 一种方法是建立一个txt文件，然后将扩展名改成mif即可。
- ◆ 另一种是在Quartus环境下，新建文件，选择建立一个mif文件。
- ◆ 利用Matlab或其他转换工具（pictomif、BMP2MIF）生成mif文件。

miqu100-100.bmp

BMP 图像



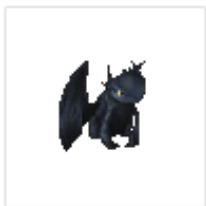
修改日期: 2023/3/21 15:34

分辨率: 100 x 100

大小: 29.3 KB

BMP2Mif.exe

应用程序



一、ROM简介

FPGA的ROM初始化：mif文件的创建与使用

◆ mif文件是在编译和仿真过程中作为存储器(RAM或ROM) 初始化输入的文件。

◆ mif文件格式为:

depth=XX; --存储深度

width=XX; --存储宽度

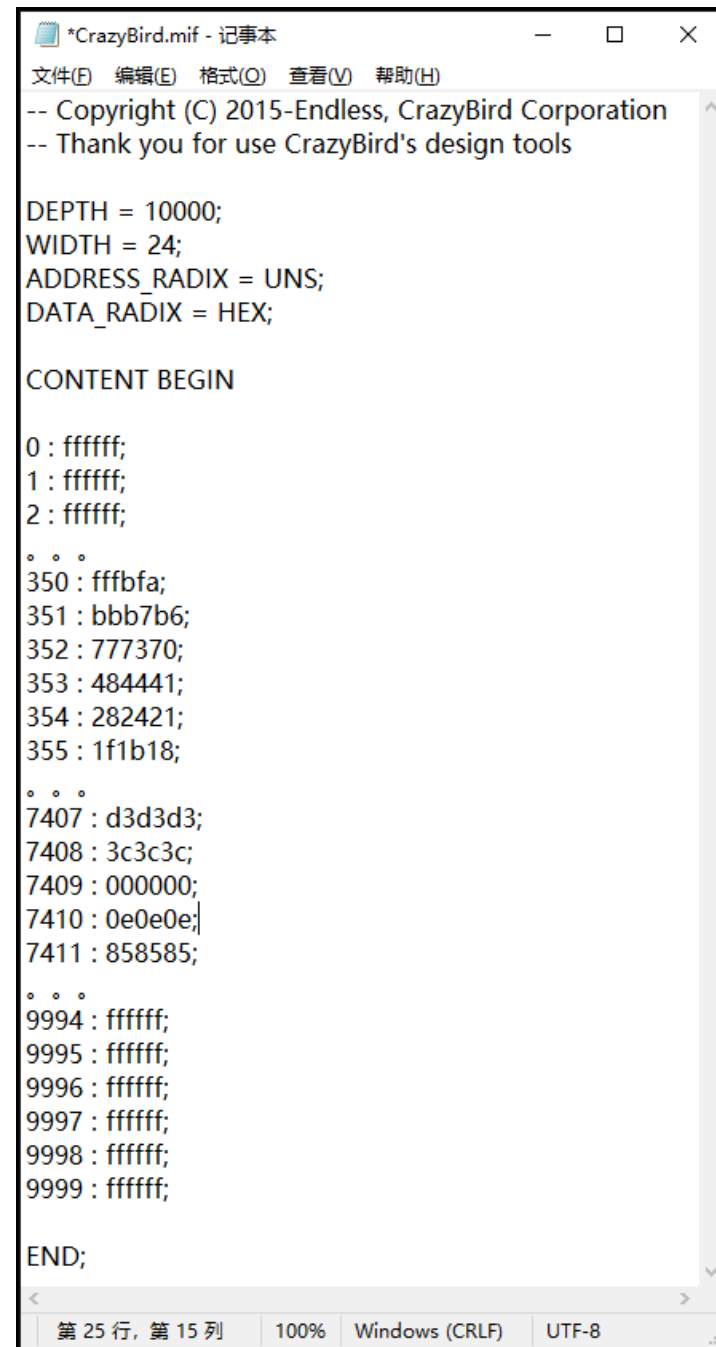
address_radix=UNS ; --地址为无符号数

data_radix=HEX; --数据为十六进制数

content begin

 XX(address):XX(data); --地址： 数据

end;



```
*CrazyBird.mif - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
-- Copyright (C) 2015-Endless, CrazyBird Corporation
-- Thank you for use CrazyBird's design tools

DEPTH = 10000;
WIDTH = 24;
ADDRESS_RADIX = UNS;
DATA_RADIX = HEX;

CONTENT BEGIN

0 : fffffff;
1 : fffffff;
2 : fffffff;

. . .
350 : fffbfa;
351 : bbb7b6;
352 : 777370;
353 : 484441;
354 : 282421;
355 : 1f1b18;

. . .
7407 : d3d3d3;
7408 : 3c3c3c;
7409 : 000000;
7410 : 0e0e0e;
7411 : 858585;

. . .
9994 : fffffff;
9995 : fffffff;
9996 : fffffff;
9997 : fffffff;
9998 : fffffff;
9999 : fffffff;

END;

第 25 行, 第 15 列 100% Windows (CRLF) UTF-8
```

二、LCD屏简介

LCD电容触摸屏模块通过40p的FPC线与实验平台连接，LCD控制器共需要29个IO口驱动（RGB888格式），电容触摸屏需要4个IO口驱动。



序号	名称	说明
1,2	VCC5	5V 电源输入引脚。
3~10	R0~R7	8 位 RED 数据线。
11	GND	地线。
12~19	G0~G7	8 位 GREEN 数据线。
20	GND	地线
21~28	B0~B7	8 位 BLUE 数据线。
29	GND	地线。
30	CLK	像素时钟。
31	HSYNC	水平同步信号。
32	VSYNC	垂直同步信号。
33	DE	数据使能信号。
34	BL	背光控制信号。
35	CS	电容触摸屏复位信号(CT_RST)
36	MOSI	电容触摸屏 IIC_SDA 信号(CT_SDA)
37	MISO	NC，电容触摸屏未用到
38	SCK	电容触摸屏 IIC_SCL 信号(CT_SCL)
39	PEN	电容触摸屏中断信号(CT_INT)
40	RESET	LCD 复位信号（低电平有效）

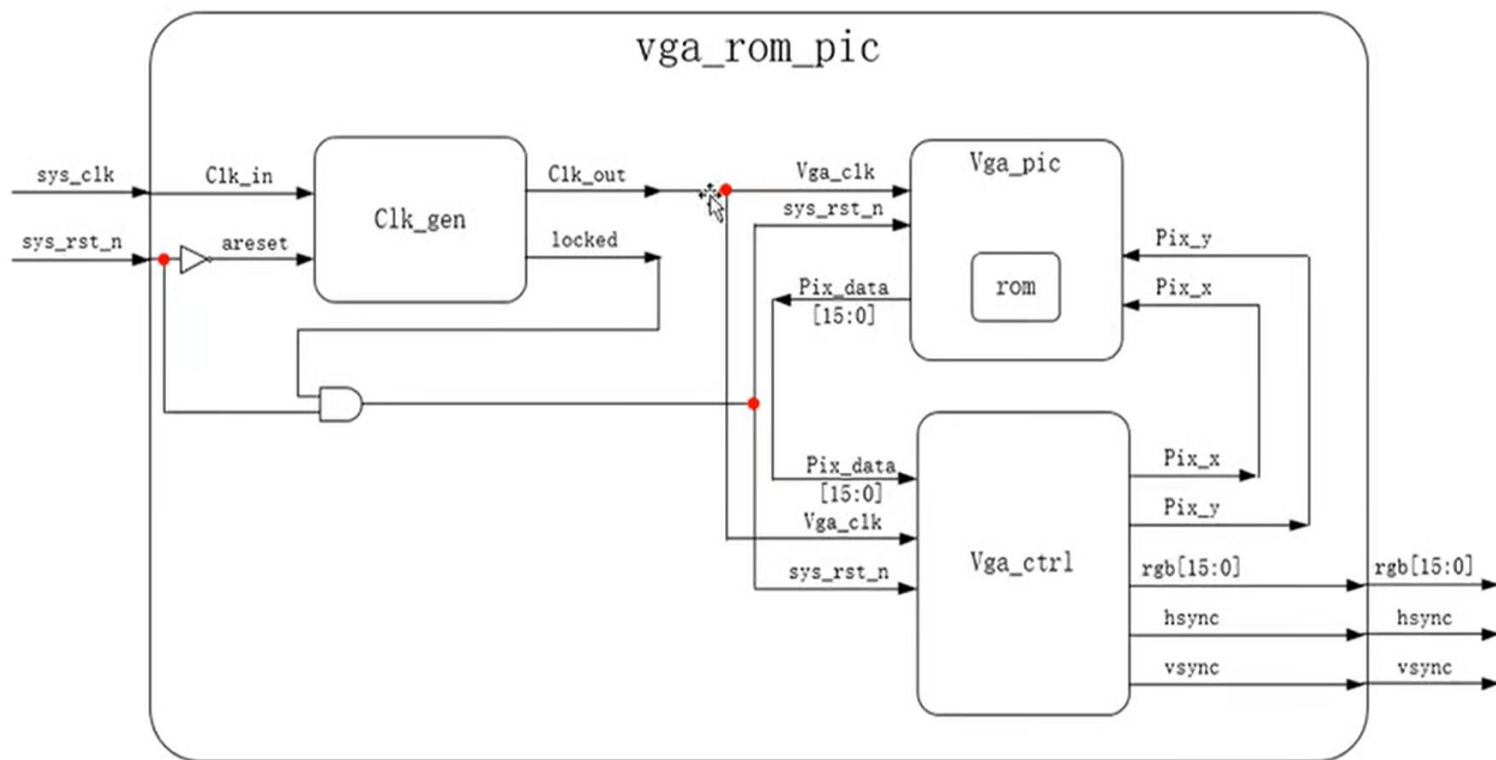
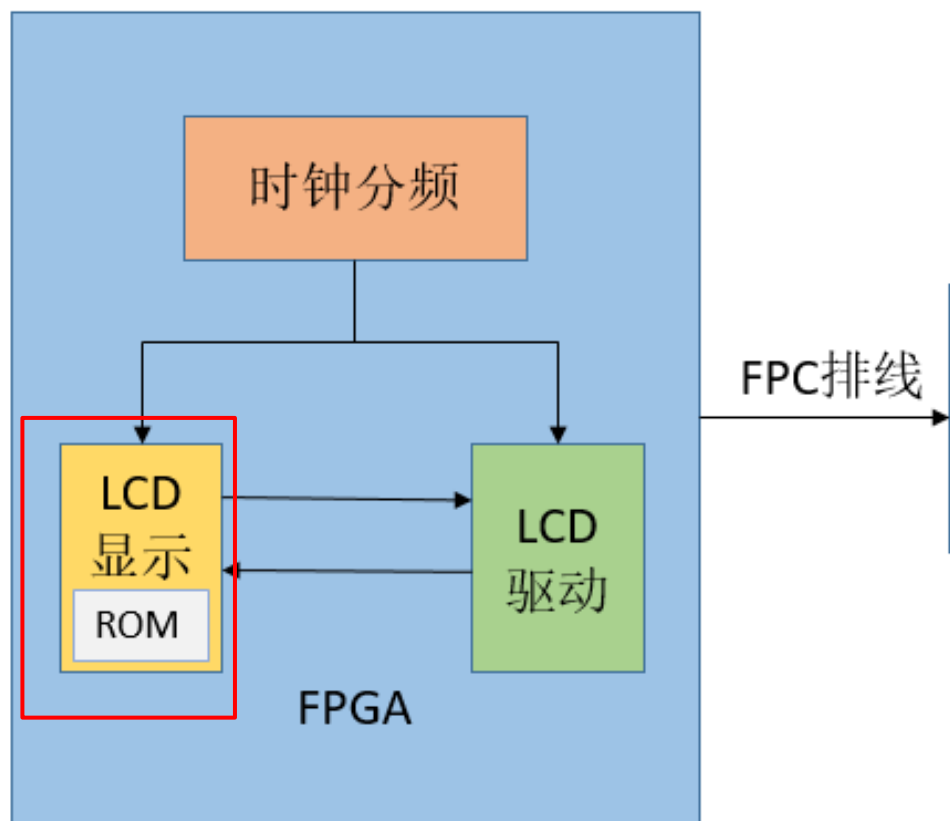
二、LCD屏简介

Parameters	Symbol	Min.	Typ.	Max.	Unit
Horizontal Display Area	thd	800			DCLK
DCLK Frequency	fcclk	26.4	33.3	46.8	MHz
One Horizontal Line	th	862	1056	1200	DCLK
HS pulse width	thpw	1	-	40	DCLK
HS Blanking	thb	46	46	46	DCLK
HS Front Porch	thfp	16	210	354	DCLK
Vertical Display Area	tvd	480			TH
VS period time	tv	510	525	650	TH
VS pulse width	tvpw	1	-	20	TH
VS Blanking	tvb	23	23	23	TH
VS Front Porch	tvfp	7	22	147	TH

LCD电容触摸屏模块采用24位并行RGB接口，其中thpw（行同步准备）、thb（行后沿）、thfp（行前沿）、tvpw（场同步准备）、tvb（场后沿）和tvfp（场前沿）六个参数，在写驱动程序时候会使用其配置LCD的时序。

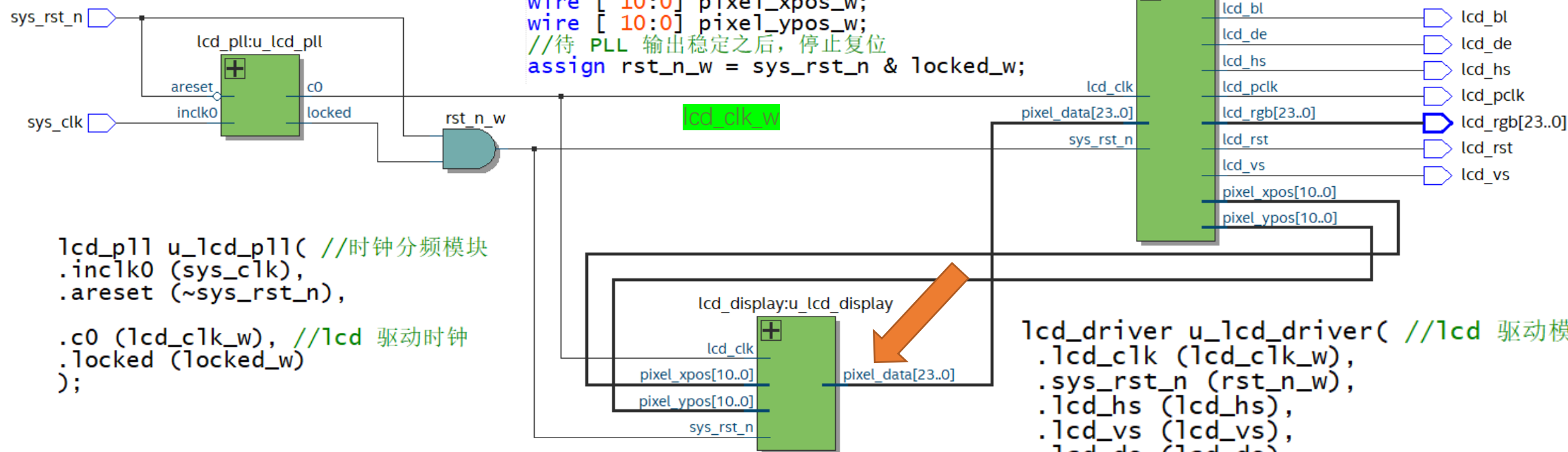
VGA 屏	R7 ↵	R6 ↵	R5 ↵	R4 ↵	R3 ↵	R2 ↵	R1 ↵	R0 ↵
FPGA 引脚 ↵	C20 ↵	R16 ↵	T16 ↵	P15 ↵	P16 ↵	T17 ↵	U17 ↵	P17 ↵
VGA 屏	G7 ↵	G6 ↵	G5 ↵	G4 ↵	G3 ↵	G2 ↵	G1 ↵	G0 ↵
FPGA 引脚 ↵	R17 ↵	N16 ↵	N17 ↵	H16 ↵	M16 ↵	G17 ↵	H17 ↵	R18 ↵
VGA 屏	B7 ↵	B6 ↵	B5 ↵	B4 ↵	B3 ↵	B2 ↵	B1 ↵	B0 ↵
FPGA 引脚 ↵	R19 ↵	K17 ↵	N18 ↵	J18 ↵	J17 ↵	G18 ↵	H18 ↵	D19 ↵
VGA 屏	LCD_BL	LCD_RST	LCD_VS	LCD_DE	LCD_HS	LCD_PCLK	↵	↵
FPGA 引脚 ↵	C15 ↵	C10 ↵	C17 ↵	D15 ↵	D17 ↵	C19 ↵	↵	↵

三、LCD屏控制器框图



四、LCD屏控制器顶层

```
wire lcd_clk_w;
wire locked_w;
wire rst_n_w;
wire [23:0] pixel_data_w;
wire [10:0] pixel_xpos_w;
wire [10:0] pixel_ypos_w;
//待 PLL 输出稳定之后, 停止复位
assign rst_n_w = sys_rst_n & locked_w;
```



```
lcd_pll u_lcd_pll( //时钟分频模块
.inclk0 (sys_clk),
.areset (~sys_rst_n),
.c0 (lcd_clk_w), //lcd 驱动时钟
.locked (locked_w)
);
```

```
lcd_driver u_lcd_driver( //lcd 驱动模块
.lcd_clk (lcd_clk_w),
.sys_rst_n (rst_n_w),
.lcd_hs (lcd_hs),
.lcd_vs (lcd_vs),
.lcd_de (lcd_de),
.lcd_rgb (lcd_rgb),
.lcd_b1 (lcd_b1),
.lcd_rst (lcd_rst),
.lcd_pclk (lcd_pclk),

.pixel_data (pixel_data_w),
.pixel_xpos (pixel_xpos_w),
.pixel_ypos (pixel_ypos_w)
);
```

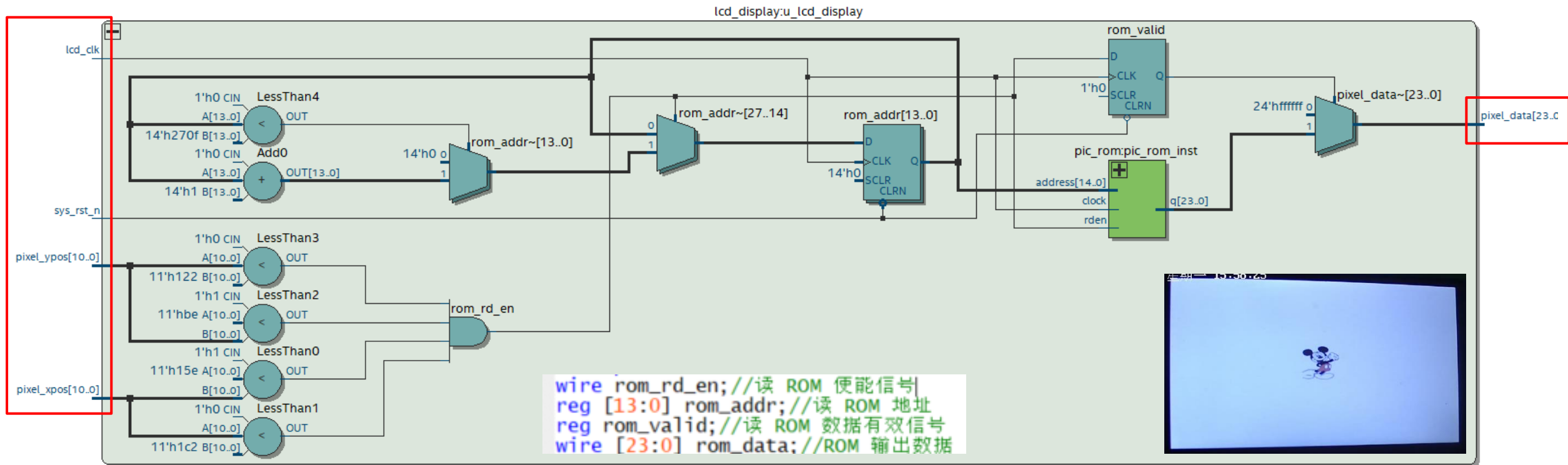
顶层模块

```
module lcd_rom_pic(
input sys_clk,
input sys_rst_n,
output lcd_hs,
output lcd_vs,
output lcd_de,
output [23:0] lcd_rgb,
output lcd_b1,
output lcd_rst,
output lcd_pclk
);
```

```
lcd_display u_lcd_display( //lcd 显示模块
.lcd_clk (lcd_clk_w),
.sys_rst_n (rst_n_w),

.pixel_xpos (pixel_xpos_w),
.pixel_ypos (pixel_ypos_w),
.pixel_data (pixel_data_w)
);
```

五、LCD屏显示模块设计-1-输入输出、参数定义、内部节点定义



```

module lcd_display(
  input lcd_clk, //lcd 驱动时钟
  input sys_rst_n, //复位信号
    
```

```

  input [10:0] pixel_xpos, //像素点横坐标
  input [10:0] pixel_ypos, //像素点纵坐标
  output [23:0] pixel_data //像素点数据
);
    
```

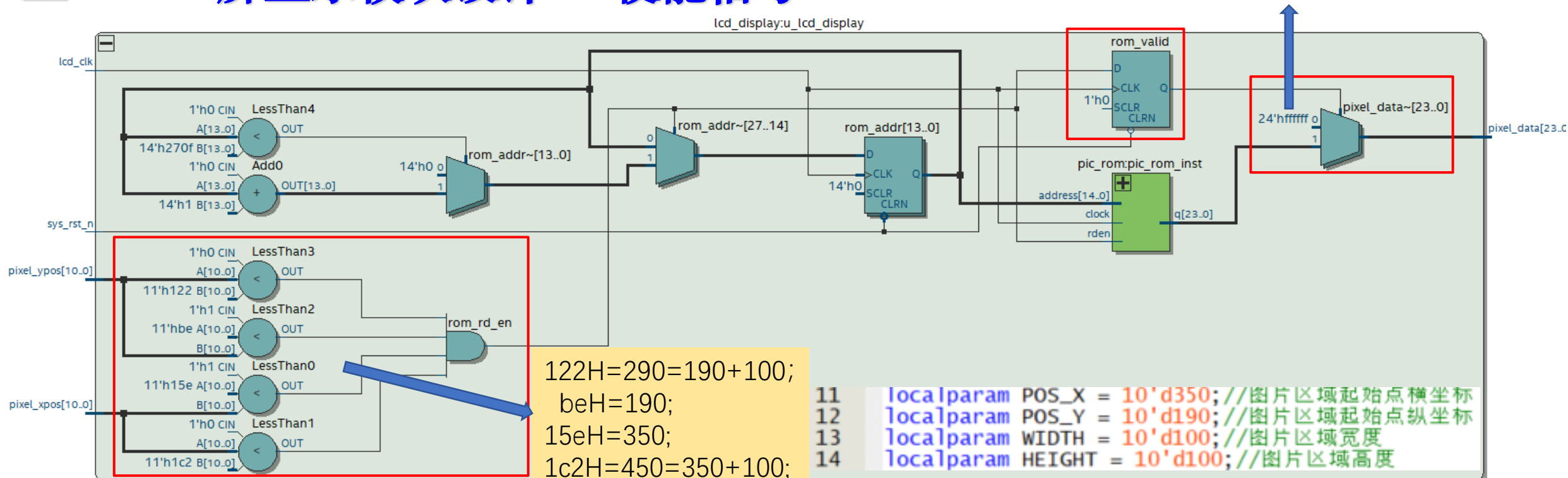
```

parameter H_DISP = 11'd800; //分辨率一行
parameter V_DISP = 11'd480; //分辨率一列
localparam POS_X = 10'd350; //图片区域起始点横坐标
localparam POS_Y = 10'd190; //图片区域起始点纵坐标
localparam WIDTH = 10'd100; //图片区域宽度
localparam HEIGHT = 10'd100; //图片区域高度
localparam TOTAL = 14'd10000; //图片区域总像素数
localparam WHITE = 24'b11111111_11111111_11111111; //图片区域背景色
    
```

基本器件：选择器、加法器、比较器、触发器

五、LCD屏显示模块设计-2-使能信号

16 localparam WHITE = 24'b11111111_11111111_11111111; //图片区域背景色



//从 ROM 中读出的图像数据有效时，将其输出显示

```
assign pixel_data = rom_valid ? rom_data : WHITE;
```

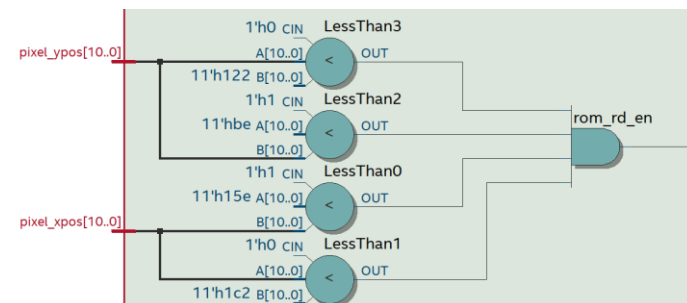
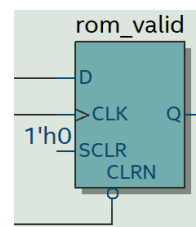
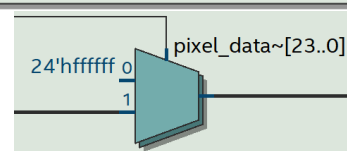
//当前像素点坐标位于图案显示区域内，ROM 使能信号拉高

```
assign rom_rd_en = (pixel_xpos >= POS_X) && (pixel_xpos < POS_X + WIDTH)
&& (pixel_ypos >= POS_Y) && (pixel_ypos < POS_Y + HEIGHT) ? 1'b1:1'b0;
```

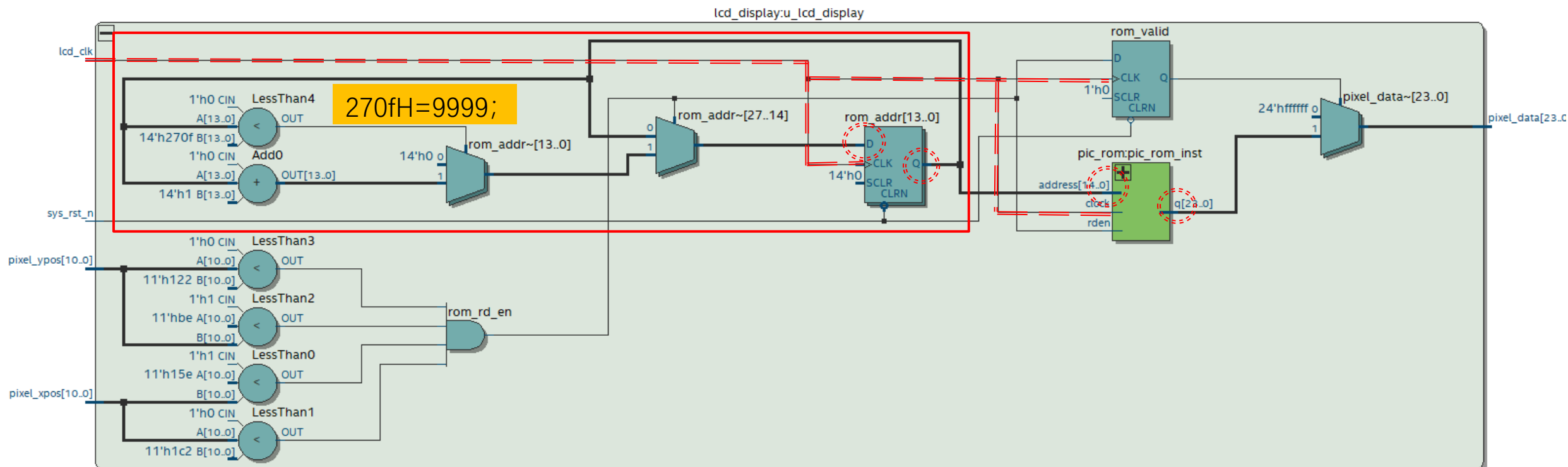
//从发出读使能信号到 ROM 输出有效数据存在一个时钟周期的延时

```
always @(posedge lcd_clk or negedge sys_rst_n) begin
if (!sys_rst_n)
rom_valid <= 1'b0;
else
rom_valid <= rom_rd_en;
end
```

见下页虚线

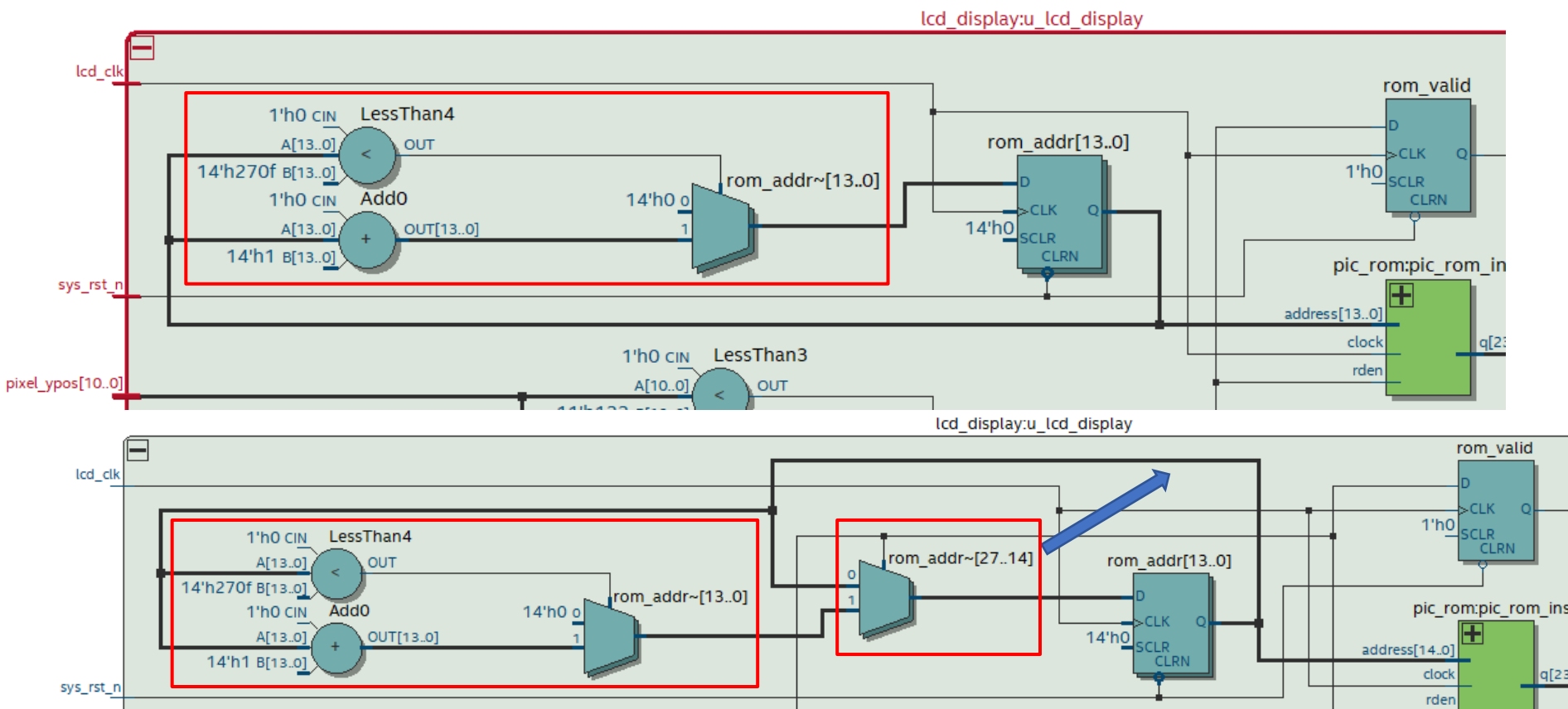


五、LCD屏显示模块设计-3-ROM地址发生控制、数据使能和数据差1clk



```
29 //控制读地址
30 always @(posedge lcd_clk or negedge sys_rst_n) begin
31     if(!sys_rst_n)
32         rom_addr <= 14'd0; //复位无效
33     else if(rom_rd_en) //在图标显示区域扫描
34         if(rom_addr < TOTAL - 1'b1) //数据取到了最大值-1 (rom偏移0开始)
35             rom_addr <= rom_addr + 1'b1; //每次读 ROM 操作后, 读地址加 1
36         else
37             rom_addr <= 1'b0; //数据取到最大, 则清零, 重新开始取像素点
38     else
39         rom_addr <= rom_addr; //当前行扫描超出显示的范围, 不用取下一个ROM单元数据, 则地址不变; 可不与
40 end
41
```


五、LCD屏显示模块设计-3-改变电路结构比较



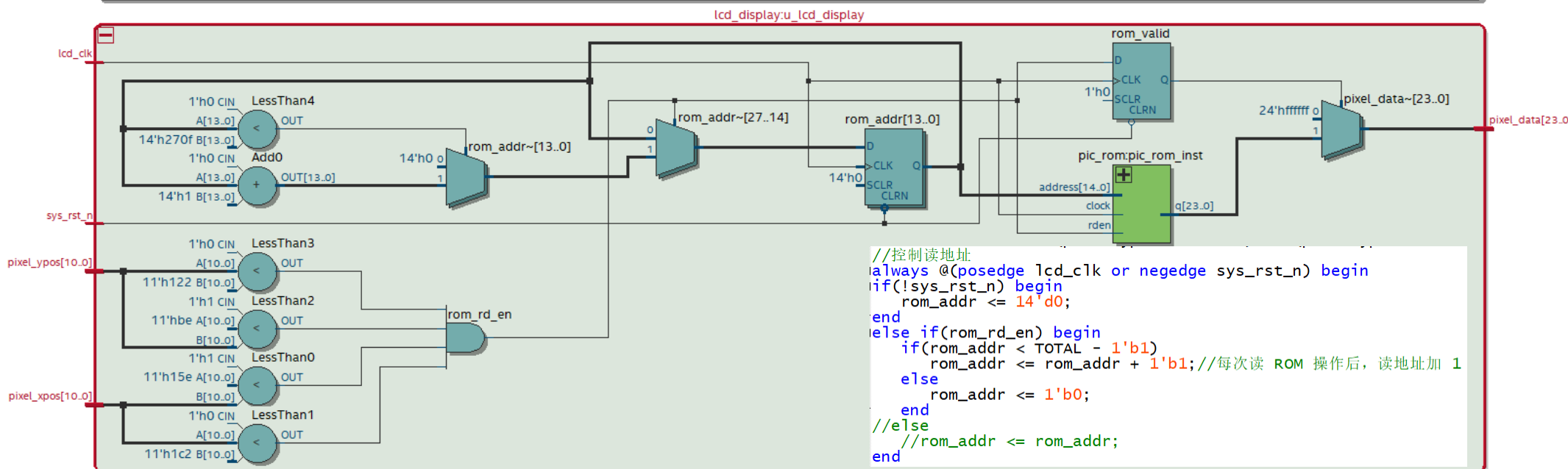
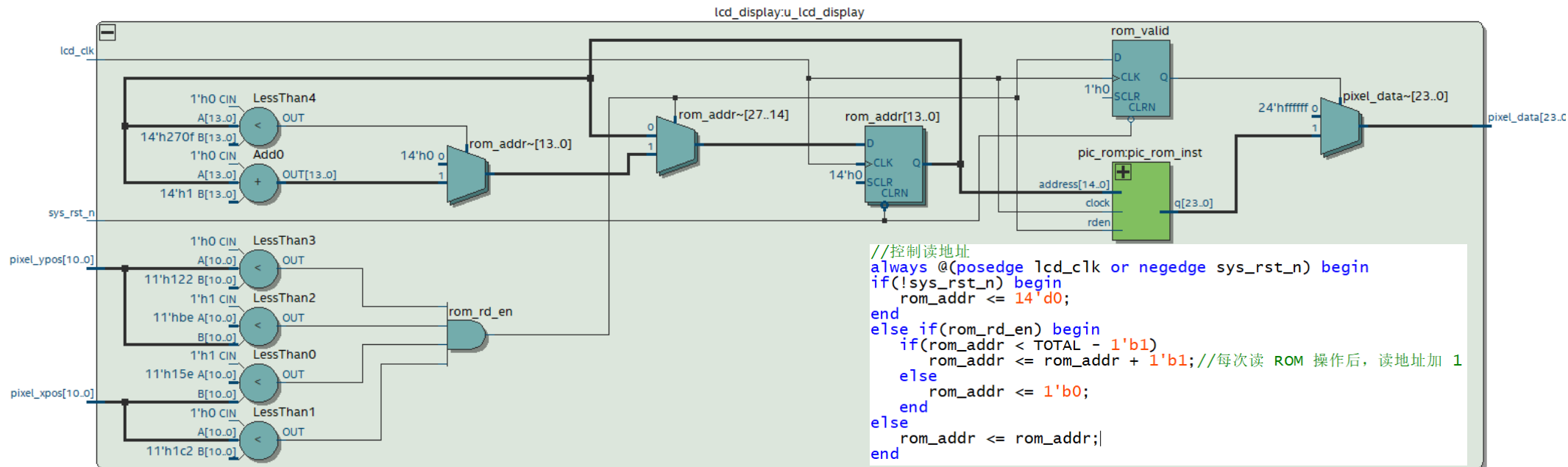
假设rom_addr的改变与rom_rd_en无关，则无论扫描显示点是否在显示的区域中，rom_addr都将增加，电路简化了，但是逻辑错误！！

注意：rom_addr[27..14]与rom_addr[13..0]并非高低位之间的关系，而是在多路分支条件下的同名影子结点。见二维码文章中的4*4路的交叉开关RTL图中的out。

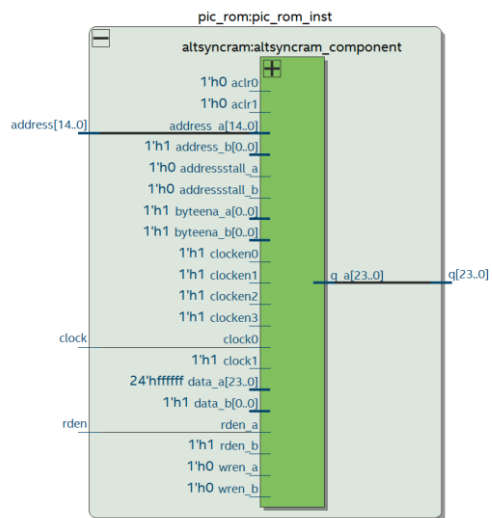
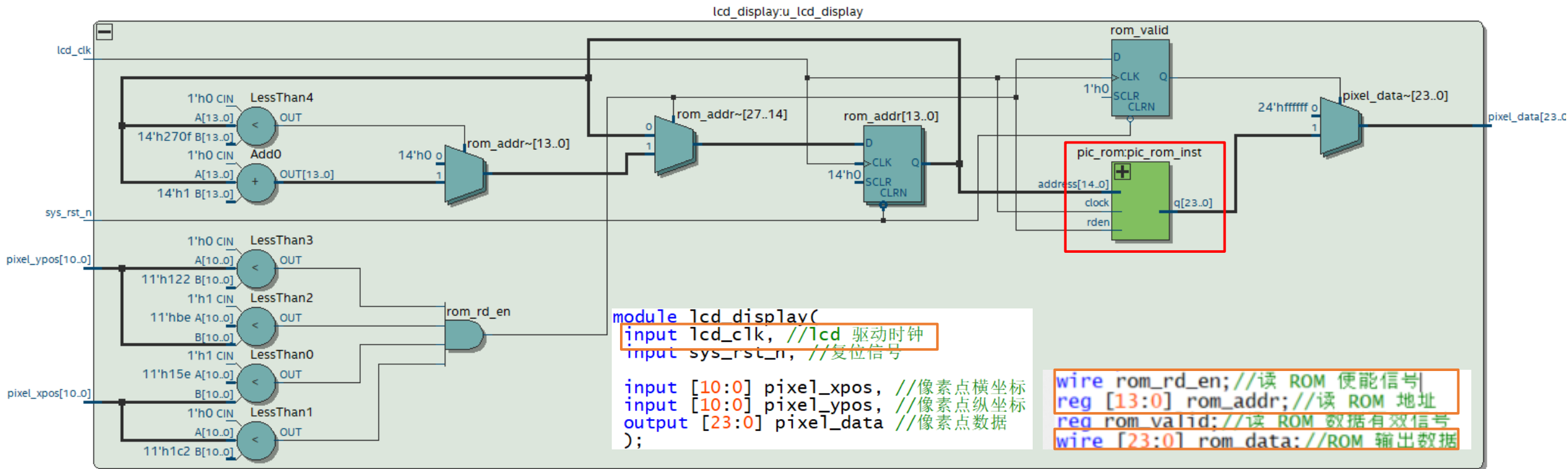
```
30 always @(posedge lcd_clk or negedge sys_rst_n) begin
31     if(!sys_rst_n)
32         rom_addr <= 14'd0;
33     else //if(rom_rd_en) begin
34         if(rom_addr < TOTAL - 1'b1)
35             rom_addr <= rom_addr + 1'b1; //每次读 ROM 操作后，读
36         else
37             rom_addr <= 1'b0; //数据取到最大，则清零，重新开始取像
38     end
```

```
29 //控制读地址
30 always @(posedge lcd_clk or negedge sys_rst_n) begin
31     if(!sys_rst_n)
32         rom_addr <= 14'd0; //复位无效
33     else if(rom_rd_en) //在图标显示区域扫描
34         if(rom_addr < TOTAL - 1'b1) //数据取到了最大值-1 (rom偏移0开始)
35             rom_addr <= rom_addr + 1'b1; //每次读 ROM 操作后，读地址加 1
36         else
37             rom_addr <= 1'b0; //数据取到最大，则清零，重新开始取像素点
38     else
39         rom_addr <= rom_addr; //当前行扫描超出显示的范围，不用取下一个ROM单元数据，则地址不变；可不用
40 end
41
```

五、LCD屏显示模块设计-3-隐含条件支路



五、LCD屏显示模块设计-4



```

module pic_rom (
    address,
    clock,
    rden,
    q);
input [14:0] address;
input clock;
input rden;
output [23:0] q;

```

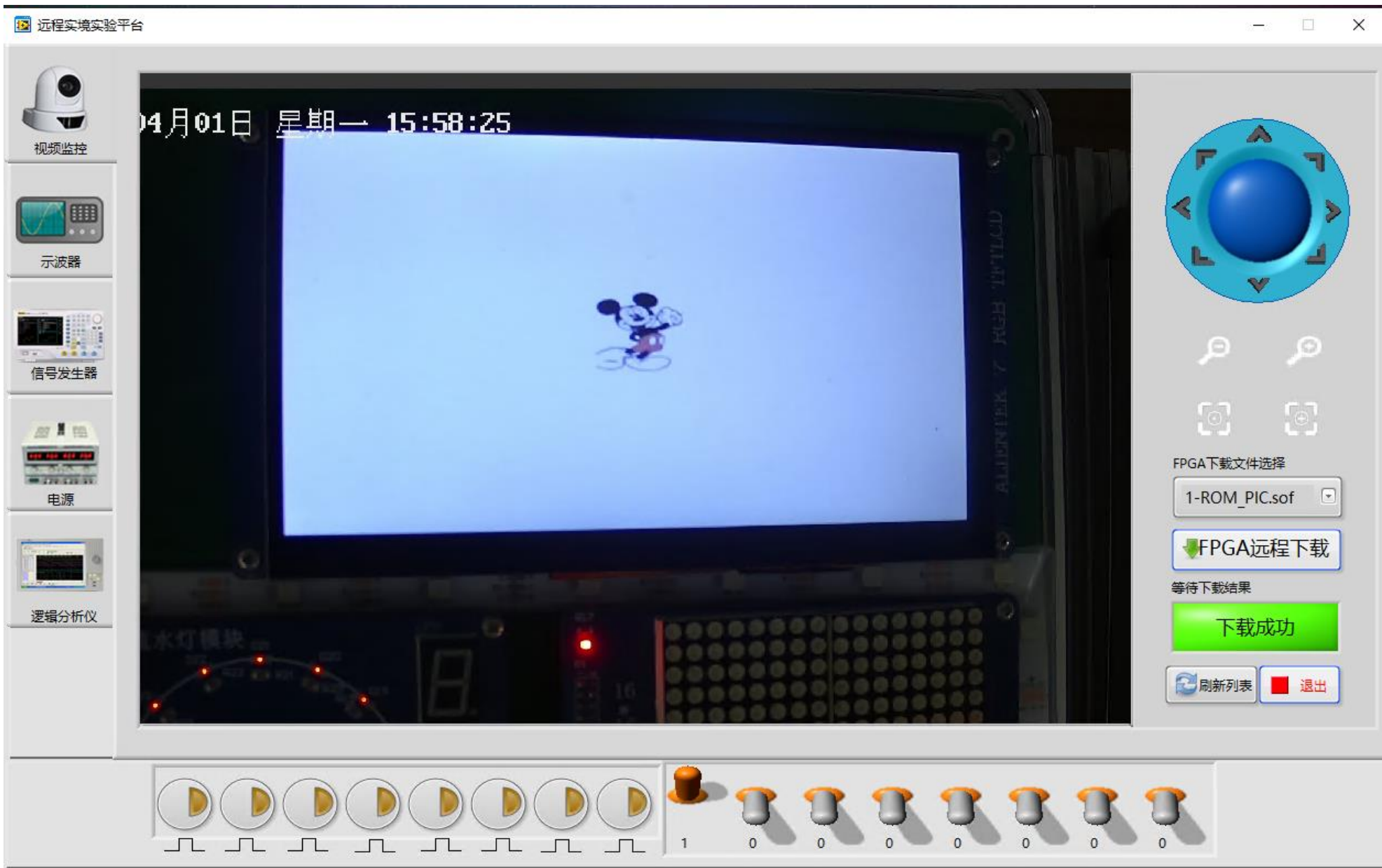
```

//通过调用 IP 核来例化 ROM
pic_rom pic_rom_inst(
    .clock (lcd_clk),
    .address (rom_addr),
    .rden (rom_rd_en),
    .q (rom_data)
);

```

ROM受rom_rd_en使能，在时钟作用下，延迟一个时钟，将地址所对应的单元内容输出

六、实验效果



七、本次实验要求含下周线上任务

完成图标显示

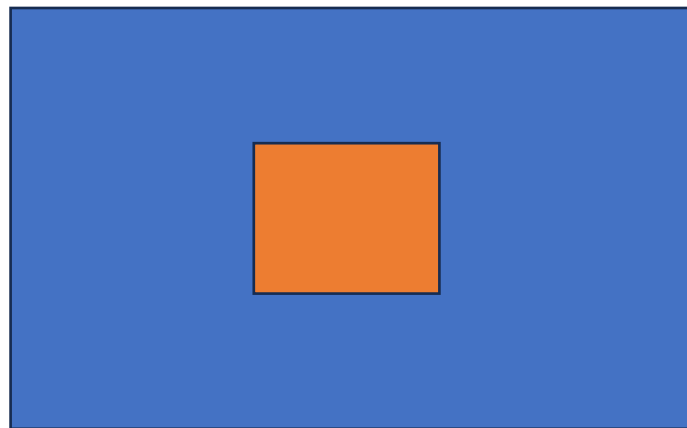
完成图标移动

完成图标灰度和二值化

固定显示

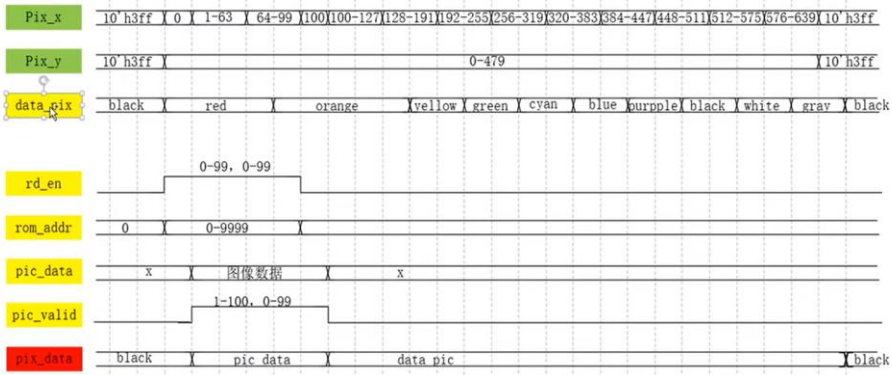
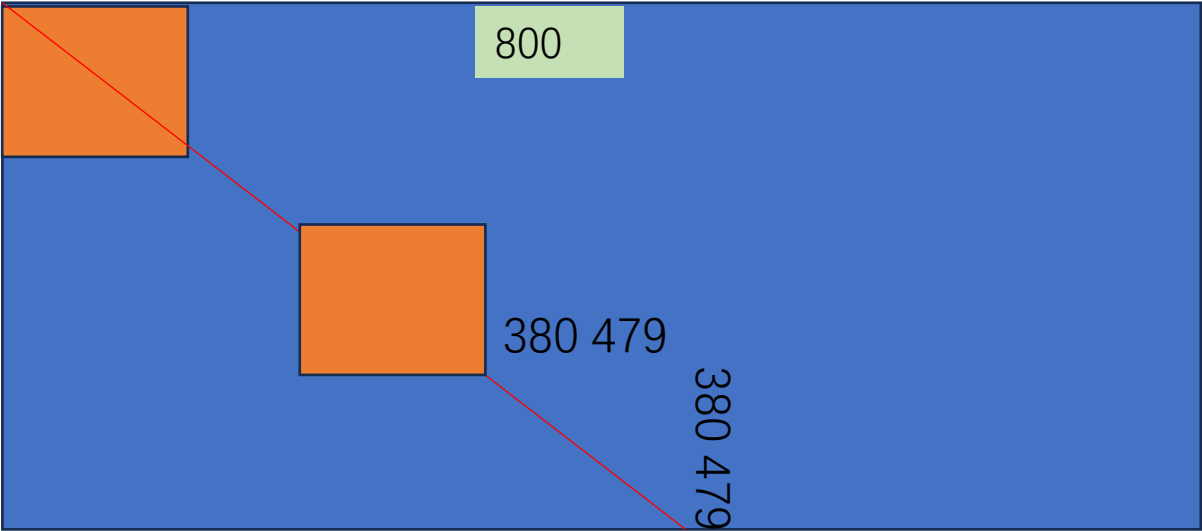
```
assign rd_en = ((pix_x >= (((H_VALID - H_PIC) / 2)) - 1'b1))  
            && ((pix_x < (((H_VALID - H_PIC) / 2) + H_PIC) - 1'b1))  
            && ((pix_y >= (((V_VALID - V_PIC) / 2)))  
            && ((pix_y < (((V_VALID - V_PIC) / 2) + V_PIC)));
```

```
parameter H_VALID = 10'd640,  
          V_VALID = 10'D480;  
  
parameter H_PIC = 10'd100,  
          V_PIC = 10'd100,  
          PIC_SIZE = 14'd10_000;
```

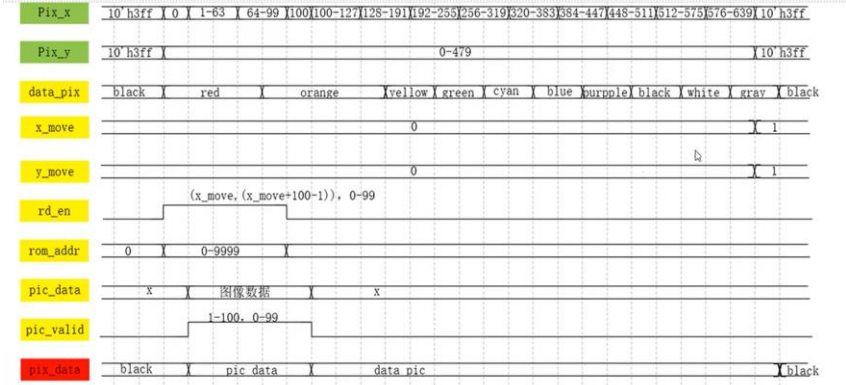


数值越大，越快；太大就不流畅

45°移动



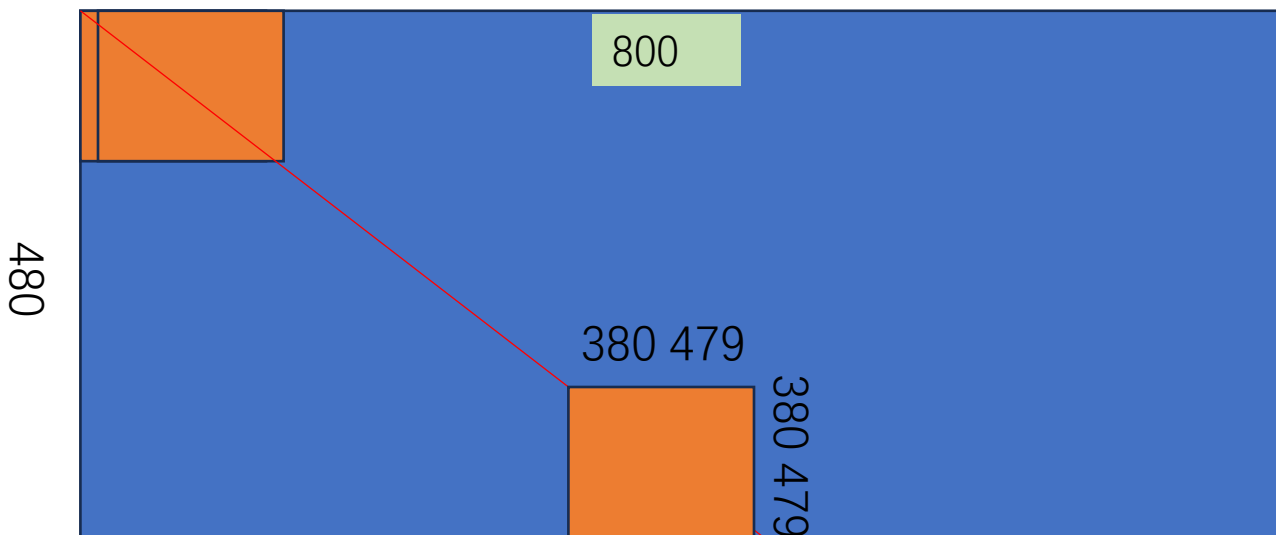
固定显示



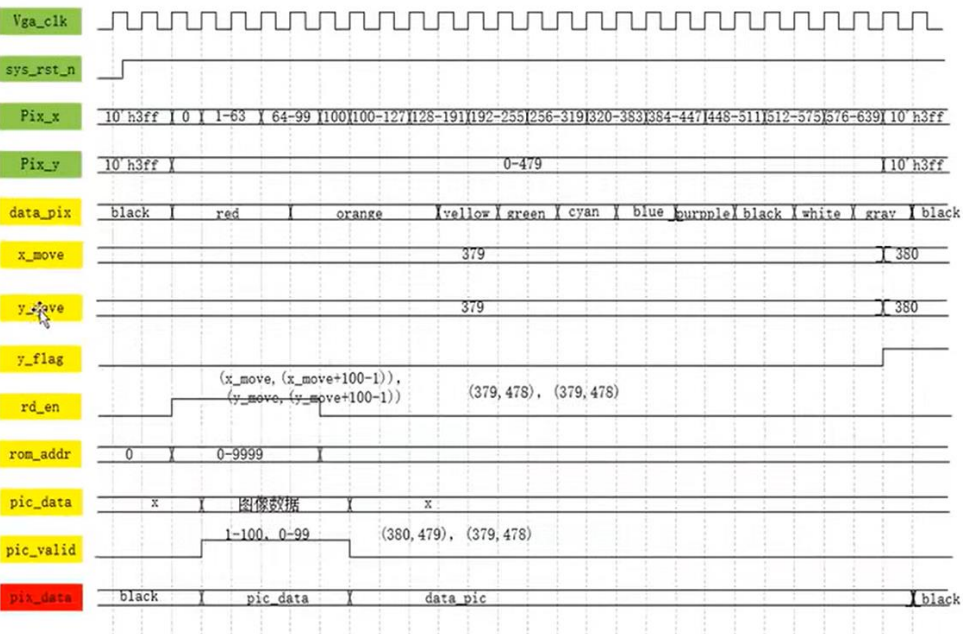
0-99, 0-99

1-100, 1-100……

触碰底框



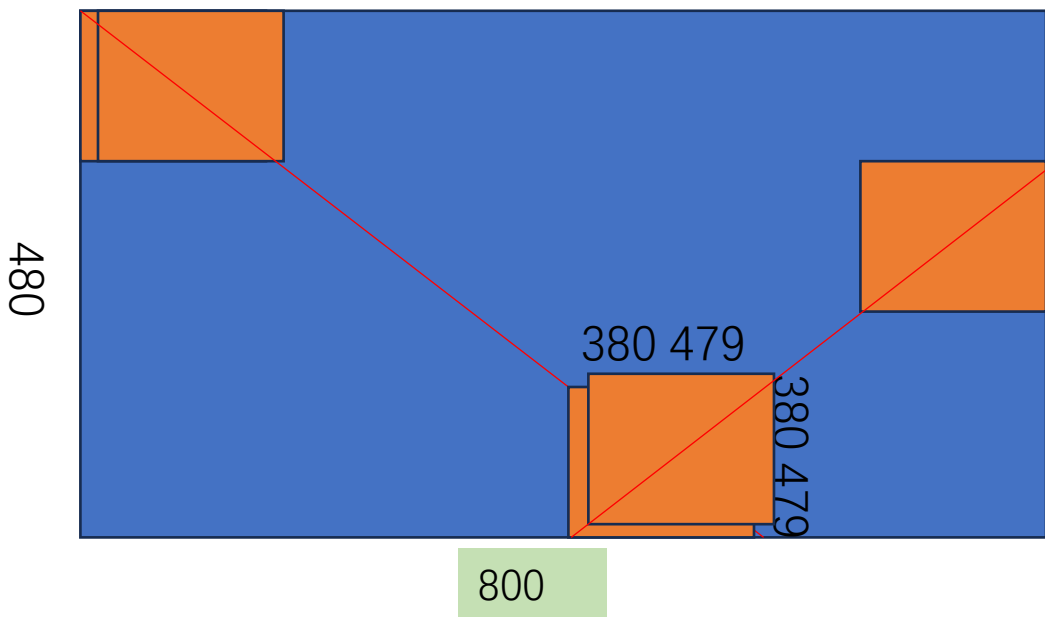
```
//y_flag: 图片上下移动标志
always@(posedge vga_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        y_flag <= 1'b0;
    else if(y_move == 0)
        y_flag <= 1'b0;
    else if((y_move == (V_VALID - W_PIC - 1'b1))
        && (pix_x == (H_VALID - 1'b1))
        && (pix_y == (V_VALID - 1'b1)))
        y_flag <= 1'b1;
```



```
//x_move: 图片横向移动量
always@(posedge vga_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        x_move <= 10'd0;
    else if((x_flag == 1'b0) && (pix_x == (H_VALID - 1'b1))
        && (pix_y == (V_VALID - 1'b1)))
        x_move <= x_move + 1'b1;
    else if((x_flag == 1'b1) && (pix_x == (H_VALID - 1'b1))
        && (pix_y == (V_VALID - 1'b1)))
        x_move <= x_move - 1'b1;
```

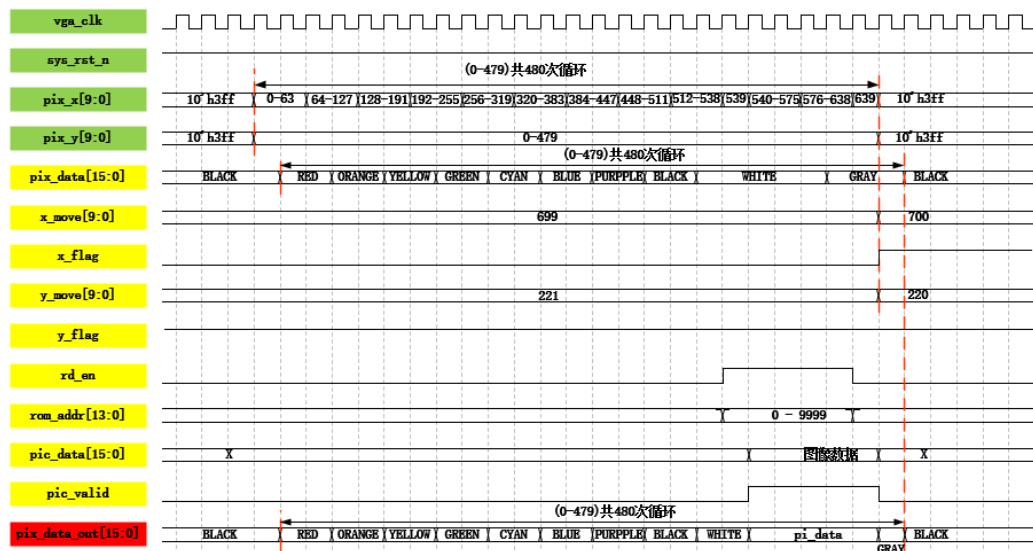
y_flag=1 @ 图像结尾

触碰右框



//x_flag: 图片左右移动标志

```
always@(posedge vga_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        x_flag <= 1'b0;
    else if(x_move == 10'd0)
        x_flag <= 1'b0;
    else if((x_move == (H_VALID - H_PIC - 1'b1))
        && (pix_x == (H_VALID - 1'b1))
        && (pix_y == (V_VALID - 1'b1)))
        x_flag <= 1'b1;
```



//y_move: 图片纵向移动量

```
always@(posedge vga_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        y_move <= 10'd0;
    else if((y_flag == 1'b0) && (pix_x == (H_VALID - 1'b1))
        && (pix_y == (V_VALID - 1'b1)))
        y_move <= y_move + 1'b1;
    else if((y_flag == 1'b1) && (pix_x == (H_VALID - 1'b1))
        && (pix_y == (V_VALID - 1'b1)))
        y_move <= y_move - 1'b1;
```


动态显示

```
//rd_en:ROM读使能
assign rd_en = (((pix_x >= (x_move))
                && (pix_x < (x_move + H_PIC)))
                && ((pix_y >= (y_move))
                && ((pix_y < (y_move + W_PIC)))));
```

```
//rd_en:ROM读使能
assign rd_en = (((pix_x >= (x_move))
                && (pix_x < (x_move + H_PIC)))
                && ((pix_y >= (y_move))
                && ((pix_y < (y_move + W_PIC)))));

//pic_valid:图片数据有效信号
always@(posedge vga_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        pic_valid <= 1'b1;
    else
        pic_valid <= rd_en;

//pix_data_out:输出VGA显示图像数据
assign pix_data_out = (pic_valid == 1'b1) ? pic_data : pix_data;
```